Integrate Zscaler with Identity Providers (IdPs) such as Azure AD, Okta, and Active Directory for seamless authentication and authorization.

TITLE: Integrate Zscaler with Identity Providers (IdPs) such as Azure AD, Okta, and Active Directory for seamless authentication and authorization.

INTRODUCTION :

Modern organizations increasingly rely on cloud services, remote work environments, and distributed networks, which have exposed the limitations of traditional security architectures based on VPNs and perimeter defenses. In conventional models, once a user successfully authenticates through a VPN, they often receive broad access to internal networks. This creates security risks such as lateral movement, unauthorized access, and increased attack surfaces**.** As cyber threats continue to evolve, organizations require more secure and flexible access control mechanisms.

To address these challenges, this project proposes the integration of the Zscaler Zero Trust Exchange platform with leading Identity Providers (IdPs) including Microsoft Entra ID (Azure AD), Okta, and Active Directory**.** This integration enables a centralized identity management system that verifies user identity before granting access to applications or resources. By adopting a Zero Trust model**,** every access request is continuously verified based on identity, context, device security, and location.

The implementation supports modern authentication standards such as SAML, OpenID Connect (OIDC), LDAP, and SCIM, allowing secure Single Sign-On (SSO), automated user provisioning, and policy-based access control. In addition to authentication integration, the project also incorporates vulnerability assessment and security testing to identify common web application threats and recommend remediation strategies based on OWASP security guidelines**.**

Overall, the project demonstrates how integrating identity management with cloud-based security platforms can significantly enhance organizational security, reduce administrative complexity, and provide a seamless and secure user access experience across enterprise applications and service

ABSTRACT :

This project focuses on integrating the Zscaler security platform with enterprise **I**dentity Providers (IdPs) such as Microsoft Entra ID (Azure AD), Okta, and on-premises Active Directory to enable secure and seamless authentication and authorization. Traditional network security models rely heavily on VPN-based access and perimeter security, which often grant broad network access after a single authentication event. This approach increases the risk of unauthorized access, lateral movement, and security breaches.

The proposed integration introduces a Zero Trust security architecture, where user identity becomes the central component of access control. By implementing Single Sign-On (SSO) and Multi-Factor Authentication (MFA) through trusted identity providers, authentication is centralized and security policies are consistently enforced across all Zscaler services. The integration leverages industry-standard protocols such as SAML, OpenID Connect (OIDC), and SCIM to enable secure authentication, automated user provisioning, and streamlined user lifecycle management..

Furthermore, the project includes vulnerability analysis and security testing using tools such as Nessus and methodologies aligned with the OWASP Top 10 framework to identify and mitigate potential web application security risks. The final outcome establishes an identity-centric security model that improves user experience, strengthens security posture, and ensures that only authenticated and authorized users can securely access applications and resources from any location.

Additionally, the integration improves overall organizational efficiency by reducing administrative overhead and eliminating the need for multiple user credential systems. Automated identity synchronization ensures that user accounts are created, updated, or removed in real time based on changes in the identity provider, thereby enhancing both security and operational management.

USE CASE SCENARIOS:

Use Case Scenario 1: Remote Faculty Access to Research Data

A research professor is working from an off-campus conference and needs secure access to sensitive research data stored on an internal server. Instead of connecting to the full campus network via VPN, ZPA grants access only to the specific research server. The professor authenticates via the university's identity provider, and access is granted based on their role.

Output:

• User identity verified: Research_Faculty_Group

• Context-aware access enforced: Access allowed only from non-campus IP ranges

• Connection restricted: Limited to specific Research File Server (R&D_Server)

• Security enforcement: Multi-Factor Authentication (MFA) required due to off-network location

• No lateral movement: User cannot access other internal systems such as HR or student records

Use Case Scenario 2: Student Access to Library Resources

A graduate student is off-campus and needs access to licensed academic journals and databases hosted internally. Instead of requiring a full VPN tunnel, ZPA provides direct access to the library application portal. The student authenticates via the identity provider, ensuring only enrolled students can access paid subscription content.

Output:

•User identity verified: Enrolled_Student_Group

• Access scope defined: Limited to Library Portal and journal databases

• Policy enforcement: Access restricted to authorized users only, preventing credential sharing

• User experience: Seamless SSO without additional VPN client complexity

• No broad network access: Student cannot navigate to administrative systems or internal servers

Use Case Scenario 3: HR Administrator Access to Payroll System

An HR administrator requires access to the payroll system to process monthly salaries. Access must be restricted to specific work hours and require step-up authentication due to the sensitivity of the data. The administrator authenticates via the identity provider, and ZPA enforces time based and MFA policies.

Output:

- User identity verified: HR_Payroll_Group
- Time-based restriction enforced: Access permitted only during business hours (8:00 AM – 6:00 PM)
- Step-up authentication: MFA triggered upon accessing the payroll application
- Connection restricted: Limited to Payroll Application Server
- Audit trail: Full session logging maintained for compliance purposes

Use Case Scenario 4: IT Helpdesk Remote Troubleshooting

An IT support technician needs to remotely troubleshoot a server issue while working from home. ZPA provides Just-in-Time (JIT) access to the specific server requiring maintenance. The technician authenticates via the identity provider, and administrative privileges are granted only for the duration of the task.

Output:

- User identity verified: IT_Helpdesk_Group
- Just-in-Time access granted: Admin rights elevated only for the active session
- Connection restricted: Limited to the specific target server (Server-X)
- Privilege boundaries enforced: No access to other infrastructure components (databases, domain controllers)
- Session monitoring: Real-time activity logging for security review

Use Case Scenario 5: Contractor Access to Project Management Tool

An external project manager contracted for a specific initiative needs access to the university's project management tool (e.g., Jira or SharePoint). Access must expire automatically upon contract completion. The contractor authenticates via the identity provider, and ZPA provisions time-limited access based on group membership synchronized from Azure AD.

Output:

- User identity verified: External_Contractor_Group
- Time-bound access policy enforced: Account deprovisioned automatically after contract end date
- Connection restricted: Limited to Project Management Application only
- Segmentation enforced: Contractor cannot discover or access internal resources like finance systems or student databases.

STAGE – 1

TITLE: Integrate Zscaler with Identity Providers (IdPs) such as Azure AD, Okta, and Active Directory for seamless authentication and authorization.

Introduction:

The project "Integrating Zscaler with Identity Providers (IdPs) such as Azure AD, Okta, and Active Directory for Seamless Authentication and Authorization" focuses on implementing a secure and efficient identity-based access management system using modern Zero Trust security principles. As organizations increasingly rely on cloud services, remote work, and distributed networks, traditional security approaches such as VPN-based access and perimeter security are no longer sufficient to protect sensitive data and applications.

This project demonstrates how the Zscaler Zero Trust Access (ZPA) platform can be integrated with widely used Identity Providers (IdPs) including Microsoft Entra ID (Azure AD), Okta, and Active Directory to enable secure authentication and policy-based access control. Through this integration, users can access internal applications and resources without exposing the internal network to the internet. Authentication is handled by the identity provider, while ZPA enforces access policies based on user identity, device posture, location, and organizational role.

The system utilizes modern authentication and provisioning protocols such as SAML, OpenID Connect (OIDC), LDAP, and SCIM to enable Single Sign-On (SSO), Multi-Factor Authentication (MFA), and automated user provisioning**.** These technologies help streamline user lifecycle management, ensuring that access is granted, modified, or revoked automatically based on user roles and organizational policies.

Additionally, the project highlights various practical scenarios including secure remote access, employee onboarding and offboarding, role-based access control, and protection against suspicious login attempts. By implementing these capabilities, the solution enhances security, reduces administrative complexity, and provides seamless user access to enterprise applications.

Another important aspect of the project is the adoption of a Zero Trust security model, which follows the principle of “never trust, always verify.” Unlike traditional security models that assume internal networks are safe, Zero Trust continuously verifies the identity and context of every access request before granting permission. This ensures that even if a network is compromised, unauthorized users cannot easily access sensitive systems or data.

The integration also improves operational efficiency for IT administrators by centralizing identity management and access control policies. Instead of managing user credentials across multiple systems, administrators can control authentication and authorization through a unified identity provider. Automated synchronization and policy enforcement reduce manual configuration and minimize the risk of human errors.

Furthermore, the project strengthens security monitoring and auditing capabilities by providing detailed logs of user authentication, access requests, and application usage. These logs help organizations detect suspicious activities, investigate potential security incidents, and maintain compliance with industry standards and regulatory requirements.

Overall, the implementation of Zscaler with enterprise identity providers creates a scalable, secure, and user-friendly access management solution. It supports modern digital workplaces by enabling secure access to applications from any location while maintaining strict security policies. This approach helps organizations protect sensitive resources, improve user productivity, and maintain strong cybersecurity posture in an evolving threat landscape.

Team Details

COLLEGE NAME: KPR COLLEGE OF ARTS SCIENCE AND RESEARCH

DEPARTMENT: B.Sc. COMPUTER SCIENCE WITH DATA ANALYTICS

LIST OF TEAMMATES :

S.NO	NAME	COLLEGE	CONTACT
01	DHARANISH K T	KPR College of Arts Science and Research	23bda013@kprcas.ac.in
02	JASEEM ATHRAPPIL	KPR College of Arts Science and Research	23bda023@kprcas.ac.in
02	KAVINN ISHWARYA A	KPR College of Arts Science and Research	23bda026@kprcas.ac.in
04	KOWSALYA M	KPR College of Arts Science and Research	23bda028@kprcas.ac.in
05	KUMARAN T S	KPR College of Arts Science and Research	23bda029@kprcas.ac.in

LIST OF VULNERABILITY TABLE :

S.No	Category Name	Vulnerability Name	CWE/CVE - No
01.	Broken Access Control	Improper Access Control	CWE-284
02.	Security Misconfiguration	Password in Configuration File	CWE-260
03.	Software Supply Chain Failures	Use of Obsolete Function	CWE-447
04.	Cryptographic Failures	Improper Following of a Certificate's Chain of Trust	CWE-296
05.	Injection	Improper Input Validation	CWE-20
06.	Insecure Design	External Control of File Name or Path	CWE-73
07.	Authentication Failures	Use of Hard-coded Password	CWE-259
08.	Software or Data Integrity Failures	Untrusted Search Path	CWE-426
09.	Security Logging & Alerting Failures	Improper Output Neutralization for Logs	CWE-117
10.	Mishandling of Exceptional Conditions	Failure to Handle Missing Parameter	CWE-234

REPORT:-

1.

Vulnerability Name:- Improper Access Control

CWE/CVE : - CWE-284

OWASP/SANS Category:- A01:2025 Broken Access Control

Description:- Improper Access Control occurs when an application fails to properly enforce restrictions on what authenticated users are allowed to do. This vulnerability allows unauthorized users to access protected resources, modify sensitive data, or perform actions beyond their permitted privileges. It may occur due to misconfigured permissions, missing authorization checks, or improper role-based access control implementation.

Business Impact::-

• Unauthorized users may gain access to confidential or sensitive information

• Attackers could modify, delete, or manipulate critical organizational data

• Exposure of internal systems could lead to further security breaches

• Violation of regulatory compliance requirements such as GDPR, PCI-DSS, or HIPAA

•Loss of organizational reputation and customer trust

• Potential financial losses due to data breaches, legal penalties, or service disruptions

```
def makeNewUserDir(username):
  if invalidUsername(username):

    #avoid CWE-22 and CWE-78
    print('Usernames cannot contain invalid characters')
    return False

  try:
    raisePrivileges()
    os.mkdir('/home/' + username)
    lowerPrivileges()

  except OSError:
    print('Unable to create new user directory for user:' + username)
    return False

  return True
```

2.

Vulnerability Name:-Password in Configuration File

CWE/CVE : - CWE-260

OWASP/SANS Category:- A02:2025 Security Misconfiguration

Description:- This vulnerability occurs when an application stores sensitive credentials such as passwords in configuration files in plain text or in an insecure format. If these configuration files are accessible to unauthorized users or attackers, they may retrieve the stored credentials and gain unauthorized access to systems, databases, or services. This weakness often arises due to poor security practices in application configuration and credential management.

Business Impact::-

•Unauthorized users may obtain system or database credentials
• Attackers could gain privileged access to internal applications or infrastructure
• Sensitive data may be exposed or manipulated
• Increased risk of data breaches and system compromise
• Potential violation of security policies and regulatory compliance requirements
• Financial loss and reputational damage to the organization

```
# Java Web App ResourceBundle properties file
...
webapp.ldap.username=secretUsernameWebapp
webapp.ldap.password=secretPassword
...
```

3.

Vulnerability Name:- Use of Obsolete Function

CWE/CVE : - CWE-447

OWASP/SANS Category:- A03:2025 Software Supply Chain Failures

Description:- This vulnerability occurs when an application uses outdated or obsolete functions that are no longer supported or securely maintained. In some cases, the user interface may indicate that a security feature is active, but the underlying functionality is not properly implemented or maintained. Attackers may exploit these obsolete functions to bypass security controls, access sensitive information, or compromise the system.

Business Impact::-

• Security mechanisms may fail to function as intended

• Attackers may exploit outdated functions to bypass authentication or authorization controls

• Sensitive data and system resources may become exposed

• Increased vulnerability to known exploits and cyberattacks

• System reliability and performance may be affected

• Potential financial losses and reputational damage due to security breaches

Reference	Description
CVE-2000-0127	GUI configuration tool does not enable a security option when a checkbox is selected, although that option is honored when manually set in the configuration file.
CVE-2001-0863	Router does not implement a specific keyword when it is used in an ACL, allowing filter bypass.
CVE-2001-0865	Router does not implement a specific keyword when it is used in an ACL, allowing filter bypass.
CVE-2004-0979	Web browser does not properly modify security setting when the user sets it.

4.

Vulnerability Name:- Improper Following of a Certificate's Chain of Trust

CWE/CVE : - CWE-296

OWASP/SANS Category:- A04:2025 Cryptographic Failures

Description:- This vulnerability occurs when a system fails to properly validate the chain of trust for a digital certificate. Instead of verifying that the certificate is issued by a trusted Certificate Authority (CA) and correctly linked to a trusted root certificate, the system may accept invalid or untrusted certificates. As a result, attackers may exploit this weakness to perform man-in-the-middle (MITM) attacks, intercept secure communications, or impersonate legitimate services.

Business Impact:

• Attackers may intercept sensitive communication between users and applications

• Unauthorized entities could impersonate trusted servers or services

• Confidential information such as login credentials or personal data may be exposed

• Compromise of secure communication channels (HTTPS, SSL/TLS)

• Loss of data integrity and user trust in the system

• Potential financial losses and reputational damage due to security breaches

Demonstrative Examples

Example 1

This code checks the certificate of a connected peer.

Example Language: C (bad code)

```
if ((cert = SSL_get_peer_certificate(ssl)) && host)
  foo=SSL_get_verify_result(ssl);

if ((X509_V_OK==foo) || X509_V_ERR_SELF_SIGNED_CERT_IN_CHAIN==foo))

  // certificate looks good, host can be trusted
```

In this case, because the certificate is self-signed, there was no external authority that could prove the identity of the host. The program could be communicating with a different system that is spoofing the host, e.g. by poisoning the DNS cache or using an Adversary-in-the-Middle (AITM) attack to modify the traffic from server to client.

5.

Vulnerability Name:- Improper Input Validation

CWE/CVE : - CWE-20

OWASP/SANS Category:- A05:2025 Injection

Description:- Improper Input Validation occurs when an application accepts input from users or external sources but fails to properly validate or sanitize that input before processing it. Without proper validation, attackers may insert malicious data or commands into the system. This can lead to attacks such as SQL injection, command injection, or cross-site scripting (XSS), which may compromise application functionality and data security.

Business Impact :

- Attackers may inject malicious commands or scripts into the system
- Unauthorized access to databases and sensitive information
- Data manipulation, deletion, or corruption
- System compromise leading to service disruption
- Increased risk of application exploitation and cyberattacks
- Financial loss and reputational damage due to security breaches

Example 2

This example asks the user for a height and width of an m X n game board with a maximum dimension of 100 squares.

Example Language: **C** *(bad code)*

```
...
#define MAX_DIM 100
...
/* board dimensions */

int m,n, error;
board_square_t *board;
printf("Please specify the board height: \n");
error = scanf("%d", &m);
if ( EOF == error ){
  die("No integer passed: Die evil hacker!\n");
}
printf("Please specify the board width: \n");
error = scanf("%d", &n);
if ( EOF == error ){
  die("No integer passed: Die evil hacker!\n");
}
if ( m > MAX_DIM || n > MAX_DIM ) {
  die("Value too large: Die evil hacker!\n");
}
board = (board_square_t*) malloc( m * n * sizeof(board_square_t));
...
```

While this code checks to make sure the user cannot specify large, positive integers and consume too much memory, it does not check for negative values supplied by the user. As a result, an attacker can perform a resource consumption (CWE-400) attack against this program by specifying two, large negative values that will not overflow, resulting in a very large memory allocation (CWE-789) and possibly a system crash. Alternatively, an attacker can provide very large negative values which will cause an integer overflow (CWE-190) and unexpected behavior will follow depending on how the values are treated in the remainder of the program.

6.

Vulnerability Name:- External Control of File Name or Path

CWE/CVE : - CWE-73

OWASP/SANS Category:- A06:2025 Insecure Design

Description:- This vulnerability occurs when an application allows user-supplied input to directly control or influence file names or file paths used in file system operations. Without proper validation and sanitization, attackers may manipulate the input to access unauthorized files or directories. This can lead to path traversal attacks where malicious users access sensitive system files, configuration files, or application data that should normally be restricted.

Business Impact:

- Unauthorized access to sensitive system or application files
- Exposure of confidential information such as configuration files or credentials
- Potential modification or deletion of critical system data
- Increased risk of privilege escalation and system compromise
- Disruption of application functionality or service availability
- Financial loss and reputational damage due to security breaches

Example 1

The following code uses input from an HTTP request to create a file name. The programmer has not considered the possibility that an attacker could provide a file name such as "../../tomcat/conf/server.xml", which causes the application to delete one of its own configuration files (CWE-22).

Example Language: **Java** *(bad code)*

```
String rName = request.getParameter("reportName");
File rFile = new File("/usr/local/apfr/reports/" + rName);
...
rFile.delete();
```

7.

Vulnerability Name:- Use of Hard-coded Password

CWE/CVE : - CWE-259

OWASP/SANS Category:- A07:2025 Authentication Failures

Description: This vulnerability occurs when a password is hard-coded directly into the application's source code or configuration files. These credentials are often used for authentication between systems or components. If an attacker gains access to the application code, configuration files, or binaries, they may discover the hard-coded password and use it to gain unauthorized access to systems or services.

Business Impact:

•Attackers may gain unauthorized access to internal systems or services

• Sensitive information and resources may be exposed or manipulated

• Increased risk of system compromise and privilege escalation

• Difficulty in changing credentials because they are embedded in the code

• Violation of security policies and compliance requirements

• Financial losses and reputational damage due to potential data breaches

Example 2

The following code is an example of an internal hard-coded password in the back-end:

Example Language: **C** (bad code)

```c
int VerifyAdmin(char *password) {
  if (strcmp(password, "Mew!")) {
    printf("Incorrect Password!\n");
    return(0);
  }
  printf("Entering Diagnostic Mode...\n");
  return(1);
}
```

Example Language: **Java** (bad code)

```java
int VerifyAdmin(String password) {
  if (!password.equals("Mew!")) {
    return(0);
  }
  //Diagnostic Mode
  return(1);
}
```

Every instance of this program can be placed into diagnostic mode with the same password. Even worse is the fact that if this program is distributed as a binary-only distribution, it is very difficult to change that password or disable this "functionality."

8.

Vulnerability Name:- Untrusted Search Path

CWE/CVE : - CWE-426

OWASP/SANS Category:- A08:2025 Software or Data Integrity Failures

Description:- This vulnerability occurs when an application searches for executable files or critical resources using a search path that is controlled or influenced by external inputs. If the search path includes directories that are not fully trusted, an attacker may place malicious files in those locations. When the application loads or executes these files, it may unknowingly run malicious code, leading to system compromise.

Business Impact:

- Attackers may execute malicious programs on the system
- Unauthorized code execution could compromise application security
- Sensitive data and system resources may be exposed or manipulated
- Increased risk of malware infection and system takeover
- Disruption of application functionality or service availability
- Financial loss and reputational damage due to security breaches

Example 1

This program is intended to execute a command that lists the contents of a restricted directory, then performs other actions. Assume that it runs with setuid privileges in order to bypass the permissions check by the operating system.

Example Language: **C** *(bad code)*

```c
#define DIR "/restricted/directory"

char cmd[500];
sprintf(cmd, "ls -l %480s", DIR);
/* Raise privileges to those needed for accessing DIR. */

RaisePrivileges(...);
system(cmd);
DropPrivileges(...);
...
```

This code may look harmless at first, since both the directory and the command are set to fixed values that the attacker can't control. The attacker can only see the contents for DIR, which is the intended program behavior. Finally, the programmer is also careful to limit the code that executes with raised privileges.

However, because the program does not modify the PATH environment variable, the following attack would work:

(attack code)

- The user sets the PATH to reference a directory under the attacker's control, such as "/my/dir/".
- The attacker creates a malicious program called "ls", and puts that program in /my/dir
- The user executes the program.
- When system() is executed, the shell consults the PATH to find the ls program
- The program finds the attacker's malicious program, "/my/dir/ls". It doesn't find "/bin/ls" because PATH does not contain "/bin/".
- The program executes the attacker's malicious program with the raised privileges.

9.

Vulnerability Name:- Improper Output Neutralization for Logs

CWE/CVE : - CWE-117

OWASP/SANS Category:- A09:2025 Security Logging & Alerting Failures

Description:- This vulnerability occurs when an application writes log messages using external or user-controlled input without properly sanitizing or neutralizing special characters. Attackers may inject malicious log entries or manipulate existing logs. This can lead to log forging or log injection, where attackers hide malicious activities or mislead administrators during security investigations.

Business Impact:

• Attackers may manipulate or forge log records

• Security teams may be misled during incident investigations

• Malicious activities could remain undetected due to altered logs

• Reduced effectiveness of monitoring and alerting systems

• Difficulty in forensic analysis and security auditing

• Potential compliance issues and reputational damage for the organization

Improperly sanitized log data can also affect the integrity of security monitoring systems that rely on accurate logs to detect threats. If attackers successfully inject or modify log entries, it becomes difficult for administrators to trace the origin of attacks or identify compromised systems. This may delay incident response, allowing attackers to continue exploiting the system and potentially cause greater damage to organizational resources and data.

Example 1

The following web application code attempts to read an integer value from a request object. If the parseInt call fails, then the input is logged with an error message indicating what happened.

Example Language: **Java** (bad code)

```
String val = request.getParameter("val");
try {

  int value = Integer.parseInt(val);
}
catch (NumberFormatException) {
  log.info("Failed to parse val = " + val);
}
...
```

If a user submits the string "twenty-one" for val, the following entry is logged:

- INFO: Failed to parse val=twenty-one

However, if an attacker submits the string "twenty-one%0a%0aINFO:+User+logged+out%3dbadguy", the following entry is logged:

- INFO: Failed to parse val=twenty-one
- INFO: User logged out=badguy

Clearly, attackers can use this same mechanism to insert arbitrary log entries.

10.

Vulnerability Name:- Failure to Handle Missing Parameter

CWE/CVE : - CWE-234

OWASP/SANS Category:- A10:2025 Mishandling of Exceptional Conditions

Description:- This vulnerability occurs when a function or application component fails to properly handle situations where required parameters are missing. If fewer arguments are provided than expected, the function may still attempt to process the request by reading values from unintended memory locations or the stack. This improper handling of missing parameters can lead to unpredictable behavior, application errors, or potential exploitation by attackers.

Business Impacts:

- Application may crash or behave unpredictably when required inputs are missing
- Attackers may exploit the weakness to trigger errors or unexpected system behavior
- Potential exposure of sensitive data due to improper memory handling
- Reduced reliability and stability of the application
- Increased risk of denial-of-service (DoS) conditions if functions fail repeatedly

Improper handling of missing parameters can also weaken the overall security of the application by allowing attackers to manipulate request structures or bypass validation mechanisms. This may lead to unauthorized access, data corruption, or disruption of application services. Over time, such issues can negatively affect system performance, operational continuity, and the organization's reputation for maintaining secure and reliable systems.

Example 1

The following example demonstrates the weakness.

Example Language: C (bad code)

```c
foo_funct(one, two);

void foo_funct(int one, int two, int three) {
  printf("1) %d\n2) %d\n3) %d\n", one, two, three);
}
```

Example Language: C (bad code)

```c
void some_function(int foo, ...) {
  int a[3], i;
  va_list ap;
  va_start(ap, foo);
  for (i = 0; i < sizeof(a) / sizeof(int); i++) a[i] = va_arg(ap, int);
  va_end(ap);
}

int main(int argc, char *argv[]) {
  some_function(17, 42);
}
```

This can be exploited to disclose information with no work whatsoever. In fact, each time this function is run, it will print out the next 4 bytes on the stack after the two numbers sent to it.

STAGE – 2

OVERVIEW :-

Nessus is a vulnerability scanner developed by Tenable, Inc. It is a remote scanner, meaning that it does not need to be installed on the target host to scan it. Nessus works by sending a variety of probes to the target host in order to identify vulnerabilities. These probes can include network traffic, operating system commands, and application requests.

Nessus uses a plugin-based architecture. Plugins are small pieces of code that are designed to identify specific vulnerabilities. There are over 1200 plugins available for Nessus, covering a wide range of vulnerabilities in operating systems, applications, and network devices.

When Nessus scans a host, it first uses a discovery plugin to identify the operating system and services running on the host. Once the host has been discovered, Nessus runs a series of plugins to test for vulnerabilities. For each vulnerability that Nessus finds, it provides a detailed report that includes the severity of the vulnerability, the steps to reproduce the vulnerability, and the steps to remediate the vulnerability.

Nessus can be used to scan a variety of targets, including single hosts, networks, and web applications. It can also be used to scan specific ports and services. Nessus can be configured to run scans on a schedule or manually.

Here is a step-by-step explanation of how Nessus works:

1. Nessus connects to the target host using a variety of protocols, including TCP, UDP, and ICMP.
2. Nessus sends a series of probes to the target host in order to identify its operating system and services.
3. Nessus runs a series of plugins to test for vulnerabilities.
4. Nessus generates a report that includes the severity of each vulnerability, the steps to reproduce the vulnerability, and the steps to remediate the vulnerability.

Nessus is a valuable tool for security professionals who need to identify and remediate vulnerabilities in their networks. It is easy to use and provides comprehensive reports that can be used to prioritise remediation efforts.

Here are some of the benefits of using Nessus:

- Comprehensive vulnerability coverage: Nessus has over 1200 plugins available, covering a wide range of vulnerabilities in operating systems, applications, and network devices.
- Easy to use: Nessus has a user-friendly interface that makes it easy to configure and run scans.
- Comprehensive reporting: Nessus provides detailed reports that include the severity of each vulnerability,the steps to reproduce the vulnerability, and the steps to remediate the vulnerability.
- Scalability: Nessus can be used to scan a variety of targets, including single hosts, networks, and web applications.
- Flexibility: Nessus can be configured to run scans on a schedule or manually.

Nessus is a powerful tool that can be used to improve the security of any network. It is a valuable asset for any security professional.

Target website :- http://testfire.net/

Target IP address :- 65.61.137.117

LIST OF VULNERABILITY :

S.NO	VULNERABILITY NAME	SEVERITY	PLUGINS
01.	TLS Version 1.0 Protocol Detection	Medium	104743
02.	TLS Version 1.1 Deprecated Protocol	Medium	157288
03.	TLS Version 1.1 Protocol Detection	Info	121010
04.	SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)	Low	83875
05.	ICMP Timestamp Request Remote Date Disclosure	Low	10114
06.	HSTS Missing From HTTPS Server	Info	84502
07.	SSL/TLS Recommended Cipher Suites	Info	156899
08.	SSL Certificate Signed Using Weak Hashing Algorithm (Known CA)	Info	95631
09.	SSL Root Certification Authority Certificate Information	Info	94761
10.	Nessus SYN scanner	Info	11219

REPORT:-

1.

Vulnerability Name:- TLS Version 1.0 Protocol Detection

severity : - Medium

Plugin:- 104743

Port :- 443

Description:- The remote service accepts connections encrypted using TLS 1.0. TLS 1.0 has a number of cryptographic design flaws. Modern implementations of TLS 1.0 mitigate these problems, but newer versions of TLS like 1.2 and 1.3 are designed against these flaws and should be used whenever possible.

As of March 31, 2020, Endpoints that aren't enabled for TLS 1.2 and higher will no longer function properly with major web browsers and major vendors.

PCI DSS v3.2 requires that TLS 1.0 be disabled entirely by June 30, 2018, except for POS POI terminals (and the SSL/TLS termination points to which they connect) that can be verified as not being susceptible to any known exploits.

Solution:- Enable support for TLS 1.2 and 1.3, and disable support for TLS 1.0.

Business Impact::- Using outdated encryption protocols like TLS 1.0 can expose sensitive data to interception and cryptographic attacks such as Man-in-the-Middle (MITM) attacks. Attackers may exploit these weaknesses to decrypt or manipulate secure communications, leading to data breaches, loss of customer trust, and non-compliance with security standards such as PCI DSS.

2.

Vulnerability Name:- TLS Version 1.1 Deprecated Protocol

severity : - Medium

Plugin:- 157288

Port :- 443

Description:- The remote service accepts connections encrypted using TLS 1.1. TLS 1.1 lacks support for current and recommended cipher suites. Ciphers that support encryption before MAC computation, and authenticated encryption modes such as GCM cannot be used with TLS 1.1

As of March 31, 2020, Endpoints that are not enabled for TLS 1.2 and higher will no longer

function properly with major web browsers and major vendors.

Solution:- Enable support for TLS 1.2 and/or 1.3, and disable support for TLS 1.1.

Business Impact::- Allowing deprecated protocols such as TLS 1.1 increases the risk of attackers exploiting weaknesses in outdated cryptographic mechanisms. This could potentially lead to man-in-the-middle (MITM) attacks, interception of sensitive data, or unauthorized access to confidential information transmitted between the client and the server. Additionally, continued support for deprecated protocols may result in non-compliance with modern security standards and regulatory requirements, and compatibility issues with modern browsers and applications that only support TLS 1.2 or higher.

3.

Vulnerability Name:- TLS Version 1.1 Protocol Detection

severity : - Info

Plugin:- 121010

Port :- 443

Description:-The remote service accepts connections encrypted using TLS 1.1.

TLS 1.1 lacks support for current and recommended cipher suites.

Ciphers that support encryption before MAC computation, and authenticated encryption modes such as GCM cannot be used with TLS 1.1

As of March 31, 2020, Endpoints that are not enabled for TLS 1.2 and higher will no longer function properly with major web browsers and major vendors.

Solution:- Enable support for TLS 1.2 and/or 1.3, and disable support for TLS 1.1.

Business Impact::- Allowing the use of deprecated protocols such as TLS 1.1 may expose the system to potential security risks due to weaker cryptographic protection. Attackers may exploit vulnerabilities in outdated encryption methods to intercept or manipulate sensitive data transmitted between the client and server.

4.

Vulnerability Name:- SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)

severity : - Low

Plugin:- 83875

Port :- 443

Description:- The remote host allows SSL/TLS connections with one or more Diffie-Hellman moduli less than or equal to 1024 bits. Through cryptanalysis, a third party may be able to find the shared secret in a short amount of time (depending on modulus size and attacker resources). This may allow an attacker to recover the plaintext or potentially violate the integrity of connections.

Solution:- Reconfigure the service to use a unique Diffie-Hellman moduli of 2048 bits or greater.

Business Impact::- Using weak Diffie-Hellman parameters may allow attackers to exploit cryptographic weaknesses and potentially decrypt sensitive information transmitted between clients and the server. This could lead to exposure of confidential data, session hijacking, or manipulation of communication integrity. Additionally, weak encryption configurations may result in non-compliance with modern security standards and best practices, increasing the risk of security incidents and reducing trust in the system's communication security.

5.

Vulnerability Name:- ICMP Timestamp Request Remote Date Disclosure

severity : - Low

Plugin:- 10114

Port :- 0

Description:- The remote host answers to an ICMP timestamp request. This allows an attacker to know the date that is set on the targeted machine, which may assist an unauthenticated, remote attacker in defeating time-based authentication protocols.

Timestamps returned from machines running Windows Vista / 7 / 2008 / 2008 R2 are deliberately incorrect, but usually within 1000 seconds of the actual system time.

Solution:- Filter out the ICMP timestamp requests (13), and the outgoing ICMP timestamp replies (14).

Business Impact::- Allowing ICMP timestamp responses may expose the system's internal time information to remote attackers. Although the risk is relatively low, attackers may use this information to assist in reconnaissance activities or exploit time-based authentication mechanisms. Disclosing system time information may also help attackers analyze system behavior and prepare further targeted attacks, increasing the overall exposure of the network environment.

6.

Vulnerability Name:- HSTS Missing From HTTPS Server

severity : - Info

Plugin:- 84502

Port :- 443

Description:- The remote HTTPS server is not enforcing HTTP Strict Transport Security (HSTS). HSTS is an optional response header that can be configured on the server to instruct the browser to only communicate via HTTPS. The lack of HSTS allows downgrade attacks, SSL-stripping man-in-the-middle attacks, and weakens cookie-hijacking protections.

Solution: Configure the remote web server to use HSTS.

Business Impact:

- Increased risk of man-in-the-middle (MITM) attacks during communication
- Attackers may perform SSL stripping to downgrade secure connections to HTTP
- Sensitive session cookies could be intercepted or hijacked
- Users may unknowingly access the application over insecure connections

The absence of HSTS weakens the overall transport layer security of the web application. Although the server supports HTTPS, attackers may exploit the lack of strict enforcement to intercept or manipulate network traffic. This can compromise the confidentiality and integrity of sensitive information transmitted between users and the server, potentially affecting user trust and the organization’s security posture.

7.

Vulnerability Name:- SSL/TLS Recommended Cipher Suites

severity : - Info

Plugin:- 156899

Port :- 443

Description:- The remote host has open SSL/TLS ports which advertise discouraged cipher suites. It is recommended to only enable support for the following cipher suites:

TLSv1.3:
-TLS13_AES_128_GCM_SHA256
-TLS13_AES_256_GCM_SHA384
-TLS13_CHACHA20_POLY1305_SHA256

TLSv1.2:
-ECDHE-ECDSA-AES128-GCM-SHA256
-ECDHE-RSA-AES128-GCM-SHA256
-ECDHE-ECDSA-AES256-GCM-SHA384
-ECDHE-RSA-AES256-GCM-SHA384
-ECDHE-ECDSA-CHACHA20-POLY1305
-ECDHE-RSA-CHACHA20-POLY1305

This is the recommended configuration for the vast majority of services, as it is highly secure and compatible with nearly every client released in the last five (or more) years.

Solution:- Only enable support for recommened cipher suites.

Business Impact:

- Weak cipher suites may allow attackers to exploit cryptographic vulnerabilities
- Encrypted communication between users and the server may be weakened
- Sensitive information such as login credentials or personal data could be exposed
- Increased risk of man-in-the-middle (MITM) attacks and data interception.

8.

Vulnerability Name:- SSL Certificate Signed Using Weak Hashing Algorithm (Known CA)

severity : - Info

Plugin:- 95631

Port :- 443

Description:- The remote service uses a known CA certificate in the SSL certificate chain that has been signed using a cryptographically weak hashing algorithm (e.g., MD2, MD4, MD5, or SHA1). These signature algorithms are known to be vulnerable to collision attacks (CVE-2004-2761, for example). An attacker can exploit this to generate another certificate with the same digital signature, allowing the attacker to masquerade as the affected service.

Note that this plugin reports all SSL certificate chains signed with SHA-1 that expire after January 1, 2017 as vulnerable. This is in accordance with Google's gradual sunsetting of the SHA-1 cryptographic hash algorithm.

Note that this plugin will only fire on root certificates that are known certificate authorities as listed in Tenable Community Knowledge Article 000001752. That is what differentiates this plugin from plugin 35291, which will fire on any certificate, not just known certificate authority root certificates.

Known certificate authority root certificates are inherently trusted and so any potential issues with the signature, including it being signed using a weak hashing algorithm, are not considered security issues.

Solution:- Contact the Certificate Authority to have the certificate reissued**.**

Business Impact:

• Weak hashing algorithms may reduce the strength of SSL/TLS certificate integrity
• Attackers may attempt to exploit hash collision weaknesses to forge certificates
• Potential risk of service impersonation or spoofing attacks
• Reduced trust in the authenticity of encrypted communications

9.

Vulnerability Name:- SSL Root Certification Authority Certificate Information

severity : - Info

Plugin:- 94761

Port :- 443

Description:- The remote service uses an SSL certificate chain that contains a self-signed root Certification Authority certificate at the top of the chain.

Solution:- Ensure that use of this root Certification Authority certificate complies with your organization's acceptable use and security policies.

Business Impact:

• Use of an untrusted or unauthorized root certificate may weaken the SSL/TLS trust chain
• Users may experience browser warnings or trust issues when accessing the service
• Attackers could potentially exploit weak trust configurations to impersonate services
• Reduced confidence in the authenticity of secure communications

.

Improper management of root CA certificates can affect the reliability of encrypted connections and the overall trust model of the organization’s security infrastructure. Regular verification of certificate chains helps ensure that only trusted authorities are used, maintaining secure and trusted communication between users and systems.

10.

Vulnerability Name:- Nessus SYN scanner

severity : - Info

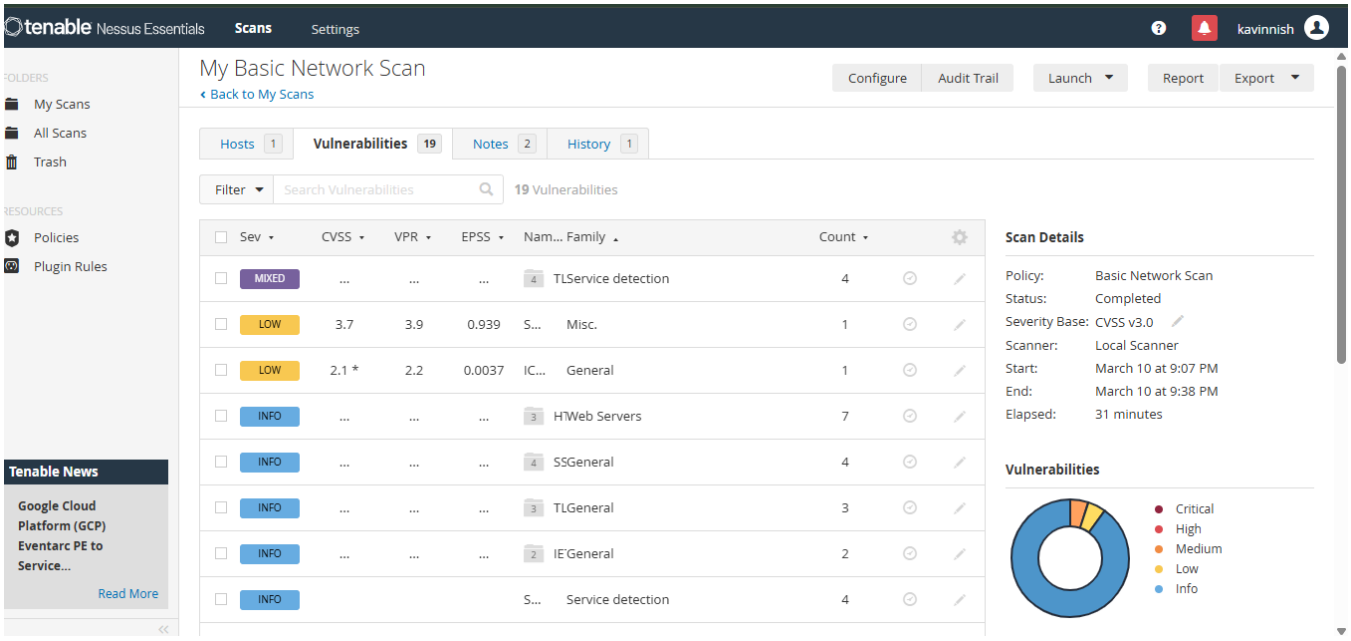
Plugin:- 11219

Port :- 80, 443, 8080

Description:- This plugin is a SYN 'half-open' port scanner. It shall be reasonably quick even against a firewalled target. Note that SYN scans are less intrusive than TCP (full connect) scans against broken services, but they might cause problems for less robust firewalls and also leave unclosed connections on the remote target, if the network is loaded.

Solution: Protect your target with an IP filter.

Business Impact:- This finding does not represent a direct security vulnerability. It simply indicates that a SYN scan technique was used during the vulnerability assessment. However, if external attackers are able to perform similar scans, they may identify open ports and services running on the system, which could assist them in performing further reconnaissance or targeted attacks.



STAGE – 3

1.SOC (Security Operations Center)

A Security Operations Center (SOC) is a centralized unit within an organization that is responsible for monitoring, detecting, analyzing, and responding to cybersecurity threats. The SOC acts as the main defense center where security professionals continuously observe network activities and system logs to identify suspicious behavior. By using advanced monitoring tools and security technologies, the SOC team ensures that potential threats are detected before they cause serious damage to the organization's systems.

In modern organizations, a large amount of security-related data is generated from sources such as servers, firewalls, network devices, and identity management systems. The SOC collects and analyzes this data to identify vulnerabilities, unauthorized access attempts, or abnormal user activities. Security analysts working in the SOC investigate alerts generated by monitoring systems and determine whether they represent real security incidents.

In this project, the SOC concept is important because the authentication activities and access requests generated through identity providers such as Azure Active Directory, Okta, and Active Directory can be monitored. By analyzing login attempts and user access patterns, the SOC can detect suspicious activities and help maintain a secure network environment.

2.SOC Cycle

The SOC cycle represents the continuous process followed by security teams to protect an organization's network and systems from cyber threats. The cycle begins with the collection of security logs from multiple sources such as network devices, servers, applications, and authentication systems. These logs contain detailed information about system activities and user behavior, which are important for detecting potential security issues.

After collecting the logs, monitoring tools analyze the data in real time to identify unusual activities or suspicious patterns. When abnormal behavior is detected, alerts are generated and security analysts begin investigating the events. The analysts carefully examine the alerts to determine whether the activity is a genuine security threat or a false alarm.

Once a threat is confirmed, appropriate response actions are taken to minimize its impact. These actions may include blocking malicious users, disabling compromised accounts, or updating security policies. After resolving the incident, the SOC team reviews the event and improves security strategies to prevent similar attacks in the future.

3.SIEM (Security Information and Event Management)

Security Information and Event Management (SIEM) is a security technology that helps organizations collect, analyze, and manage security-related data from various sources within their network. SIEM systems gather log data from devices such as servers, firewalls, network equipment, and authentication systems, and store this information in a centralized platform for analysis.

The main purpose of a SIEM system is to provide real-time visibility into network activities and identify suspicious behavior that could indicate a cyber attack. By analyzing large volumes of log data, SIEM tools can detect patterns that may represent security threats, such as repeated failed login attempts, unusual access patterns, or unauthorized system activity.

In the context of this project, SIEM can be used to monitor authentication logs from identity providers like Azure Active Directory, Okta, and Active Directory. By analyzing these logs, the system can detect possible security threats and help security teams respond quickly to potential attacks.

4.SIEM Cycle

The SIEM cycle describes the operational workflow used by SIEM systems to manage and analyze security events within an organization’s network environment. The process starts with the collection of logs from various systems and network devices. These logs contain information about system operations, user logins, and network activities.

After collecting the logs, the SIEM system processes and normalizes the data so that it can be analyzed effectively. The system then performs correlation analysis by identifying relationships between different events. For example, multiple failed login attempts followed by a successful login may indicate a possible brute force attack.

When suspicious patterns are detected, the SIEM system generates alerts for security analysts. The analysts investigate the alerts to determine whether they represent real security incidents. If a threat is confirmed, appropriate actions are taken to mitigate the risk and prevent further damage.

5.MISP (Malware Information Sharing Platform)

The Malware Information Sharing Platform (MISP) is an open-source threat intelligence platform designed to collect, store, share, and analyze information about cybersecurity threats. It allows organizations to collaborate and share information related to malware attacks, phishing campaigns, and other cyber threats.

MISP helps security teams exchange valuable threat intelligence data such as malicious IP addresses, suspicious domain names, malware file hashes, and indicators of compromise. By sharing this information with other organizations and cybersecurity communities, security teams can improve their ability to detect and prevent cyber attacks.

In environments where identity management systems and Zero Trust security platforms are used, MISP can provide additional threat intelligence. If a login request originates from an IP address that is known to be malicious, the system can block the request or trigger additional security checks to prevent unauthorized access.

6. Your College Network Information

The project is designed based on the typical network infrastructure used in a college environment. A college network usually includes multiple departments, computer laboratories, administrative offices, faculty systems, and student access networks. These systems are connected through a centralized network that provides internet access and communication between different departments.

The network infrastructure also includes internal servers that store important data such as student records, academic information, research data, and administrative documents. Because a large number of users access the network every day, it is important to implement strong authentication and access control mechanisms to protect sensitive information.

By integrating identity-based authentication systems with Zero Trust security platforms, the college network can ensure that only authorized users are allowed to access specific resources. This approach improves security and reduces the risk of unauthorized access or data breaches.

7. How You Plan to Deploy the Project in Your College Network

The deployment of this project in the college network is based on the Zero Trust security model, which ensures that every access request is verified before granting access to applications or systems. In this model, identity providers such as Active Directory or Azure Active Directory are integrated with the Zscaler platform to manage authentication and authorization.

When a user attempts to access a system or application, the identity provider verifies the user's credentials and determines whether the user has the necessary permissions. Students, faculty members, and administrative staff can securely access resources using Single Sign-On authentication with their institutional credentials.

The system also includes security monitoring features that track login activities and access requests. These logs can be analyzed using SIEM tools to detect suspicious behavior and identify potential security threats within the network environment.

8. Threat Intelligence

Threat intelligence refers to the process of collecting and analyzing information about potential cyber threats that may target an organization's systems or networks. It helps security teams understand attacker behavior, identify emerging threats, and strengthen their cybersecurity defenses.

Threat intelligence data may include information about malicious IP addresses, phishing websites, malware signatures, and known attack techniques used by cybercriminals. By analyzing this information, organizations can take proactive steps to prevent cyber attacks and improve their security posture.

In this project environment, threat intelligence can be used to detect suspicious login attempts or unusual access patterns. If a login attempt originates from a location or IP address associated with known cyber threats, additional security measures can be implemented to protect the system.

9.Conclusion

This project focuses on implementing a secure identity-based access control system using modern cybersecurity technologies such as Zscaler Zero Trust Access and enterprise identity providers like Azure Active Directory, Okta, and Active Directory. The system ensures that only authenticated and authorized users can access specific applications or resources within the network.

By implementing authentication mechanisms such as Single Sign-On and Multi-Factor Authentication, the project improves security while simplifying the user login process. The integration of security monitoring systems such as SOC operations, SIEM analysis, and threat intelligence further enhances the ability to detect and respond to cyber threats.

Overall, the project demonstrates how modern identity-based security models can replace traditional network security approaches and provide better protection for organizational systems and data.

10.Future Scope

The concepts implemented in this project can be further improved in the future by integrating advanced technologies such as artificial intelligence and machine learning for automated threat detection. These technologies can help security systems analyze large amounts of data more efficiently and identify complex attack patterns.

Future improvements may also include the integration of advanced SIEM platforms and automated incident response systems. These tools can reduce the time required to detect and respond to cyber threats, thereby improving the overall security posture of the organization.

Additionally, the system can be expanded by integrating endpoint security tools, cloud security monitoring solutions, and advanced threat intelligence platforms. These enhancements will help organizations build stronger cybersecurity defenses and protect their networks from evolving threats.

11.Tools Used

Several tools and technologies were used in this project to implement secure identity-based access control and security monitoring. The Zscaler Zero Trust Access platform was used to provide secure application access without exposing internal systems directly to the internet. Identity providers such as Microsoft Entra ID (Azure Active Directory), Okta, and Active Directory were used for authentication and user identity management.

Vulnerability assessment tools such as Nessus were used to identify potential security weaknesses in systems and networks. Security monitoring tools and SIEM platforms were used to collect and analyze logs generated from authentication systems and network devices.

These tools work together to create a secure and efficient system that protects organizational resources while allowing authorized users to access applications safely.

12.Tools Explored

During the research and development phase of this project, several cybersecurity tools were explored to understand their capabilities and applications. Security Information and Event Management platforms such as Splunk and IBM QRadar were studied to learn how centralized security monitoring and threat detection systems work.

The ELK Stack, which includes Elasticsearch, Logstash, and Kibana, was explored as an open-source solution for log management and data analysis. The Malware Information Sharing Platform was also studied to understand how threat intelligence information can be shared between organizations.

Other tools such as Wireshark were explored for network traffic analysis, and Burp Suite was studied for web application security testing. Exploring these tools provided valuable insights into various cybersecurity technologies used to protect modern network environments.